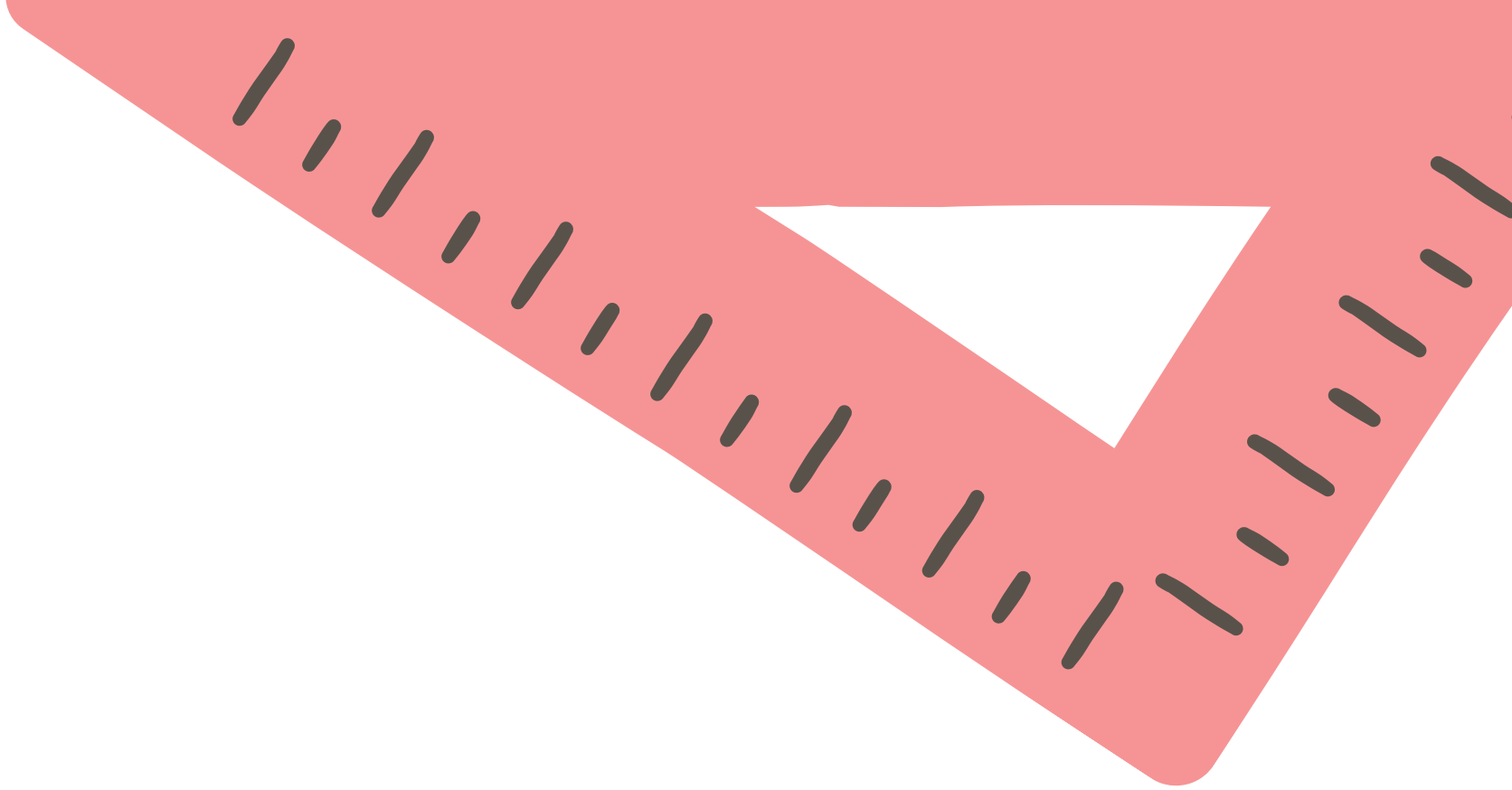
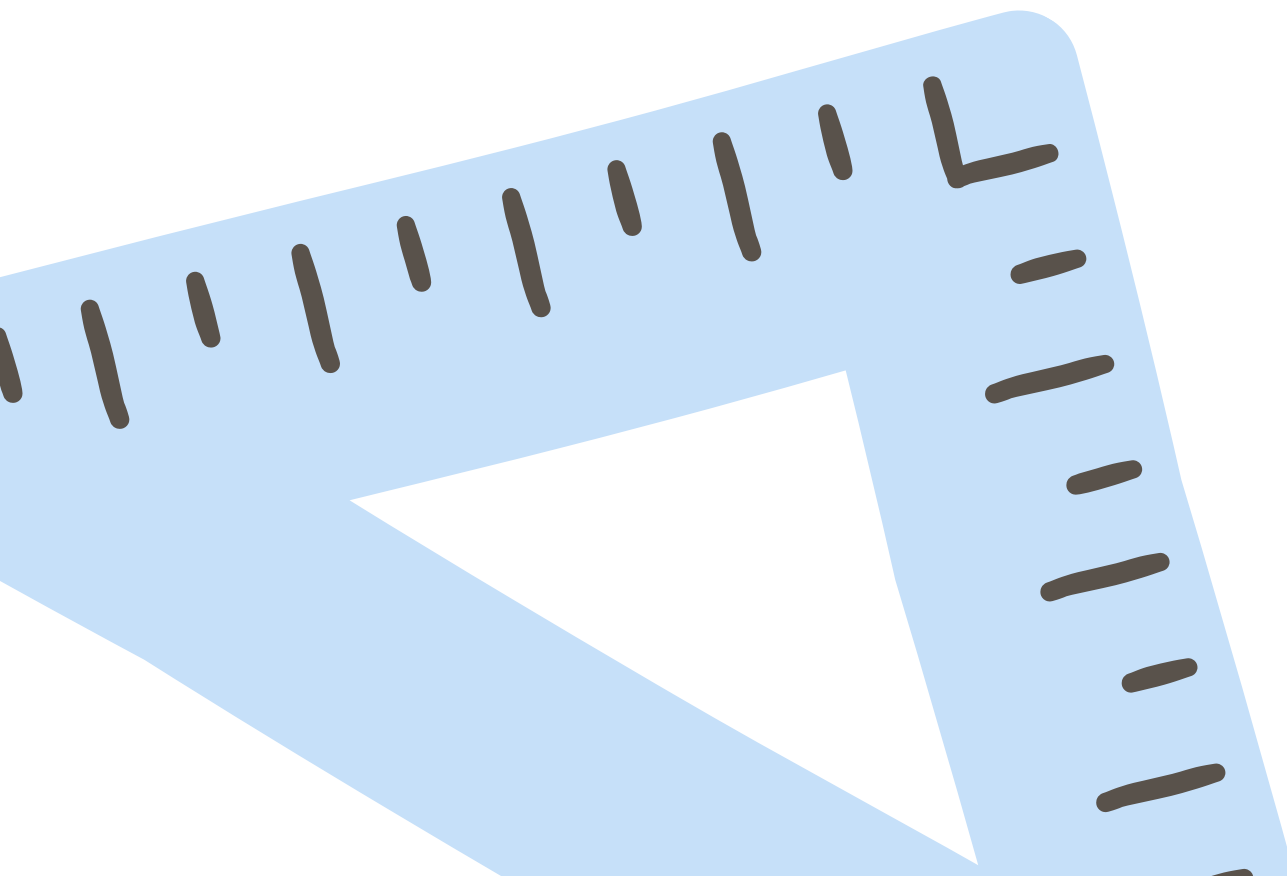




20 NEWSGROUPS SEARCH ENGINE WITH ADVANCED RANKING METHODS



1155175123

Tsoi Ming Hon



PROJECT OVERVIEW

- Project: 20 Newsgroups Search Engine with Advanced Ranking Methods"
- Dataset: 20 Newsgroups – 18,846 Usenet posts across 20 categories (e.g., comp.graphics, rec.sport.hockey)

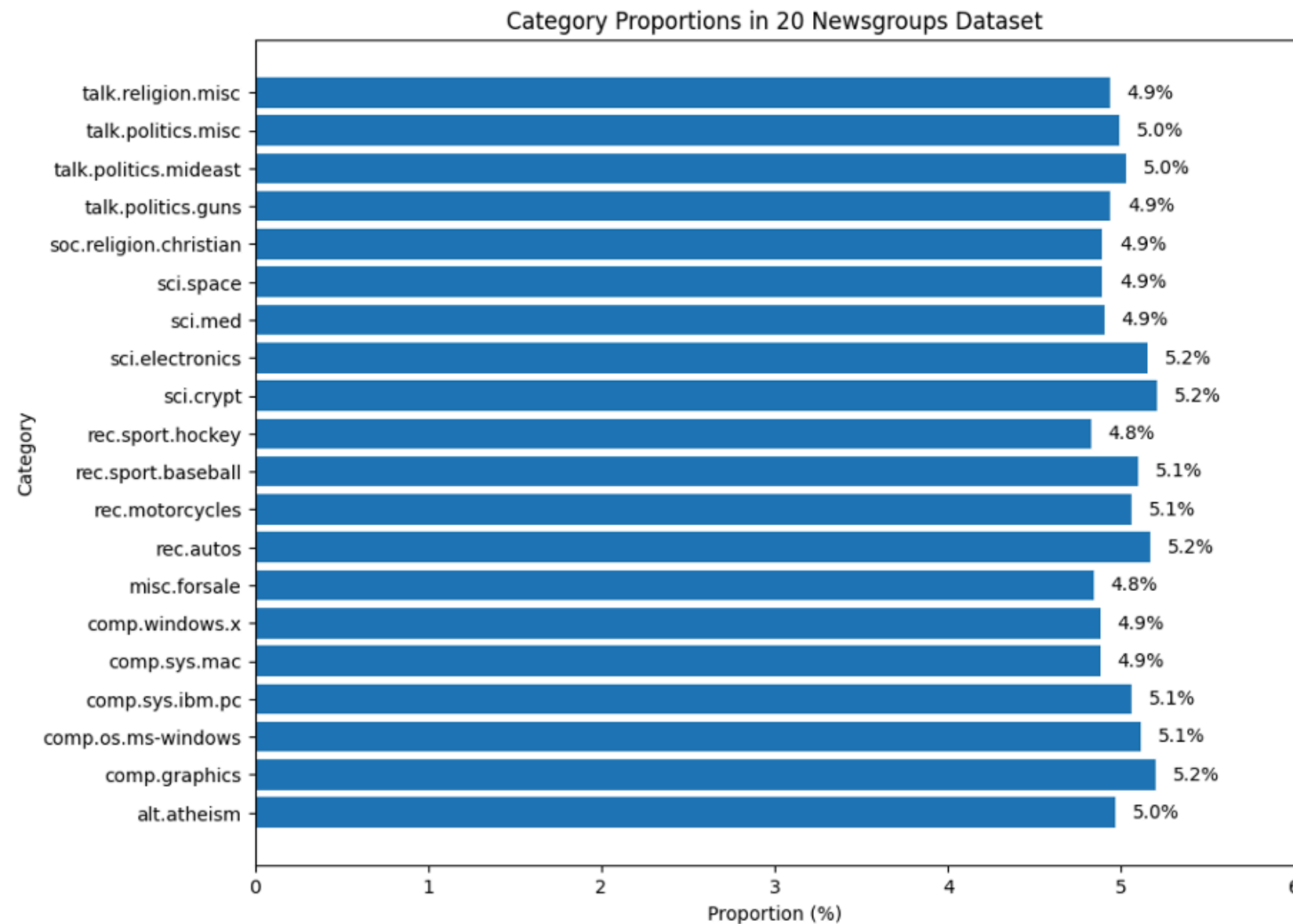
20 Newsgroups

nearly) evenly across 20 different newsgroups. To the best of my knowledge, it was the largest and most balanced data set for experiments in text applications of machine learning techniques, such as

newsgroups are very closely related to each other (e.g. **comp.sys.ibm.pc.hardware** and **comp.sys.ibm.pc.hardware.misc** matter:

comp.graphics comp.os.ms-windows.misc comp.sys.ibm.pc.hardware comp.sys.mac.hardware comp.windows.x	rec.autos rec.motorcycles rec.sport.baseball rec.sport.hockey	sci.crypt sci.electronics sci.med sci.space
misc.forsale	talk.politics.misc talk.politics.guns talk.politics.mideast	talk.religion.misc alt.atheism soc.religion.christian

WHY THIS PROJECT?



Each category constitutes 4.8%-5.2% of the dataset, confirming balance and ensuring fairness in retrieval.

- Explores how search engines facilitate information access in networked environments.
- Relevance: 20 Newsgroups is a benchmark dataset in information retrieval (IR) education.

OBJECTIVES

- Develop a search engine supporting three ranking methods:"

1. TF-IDF

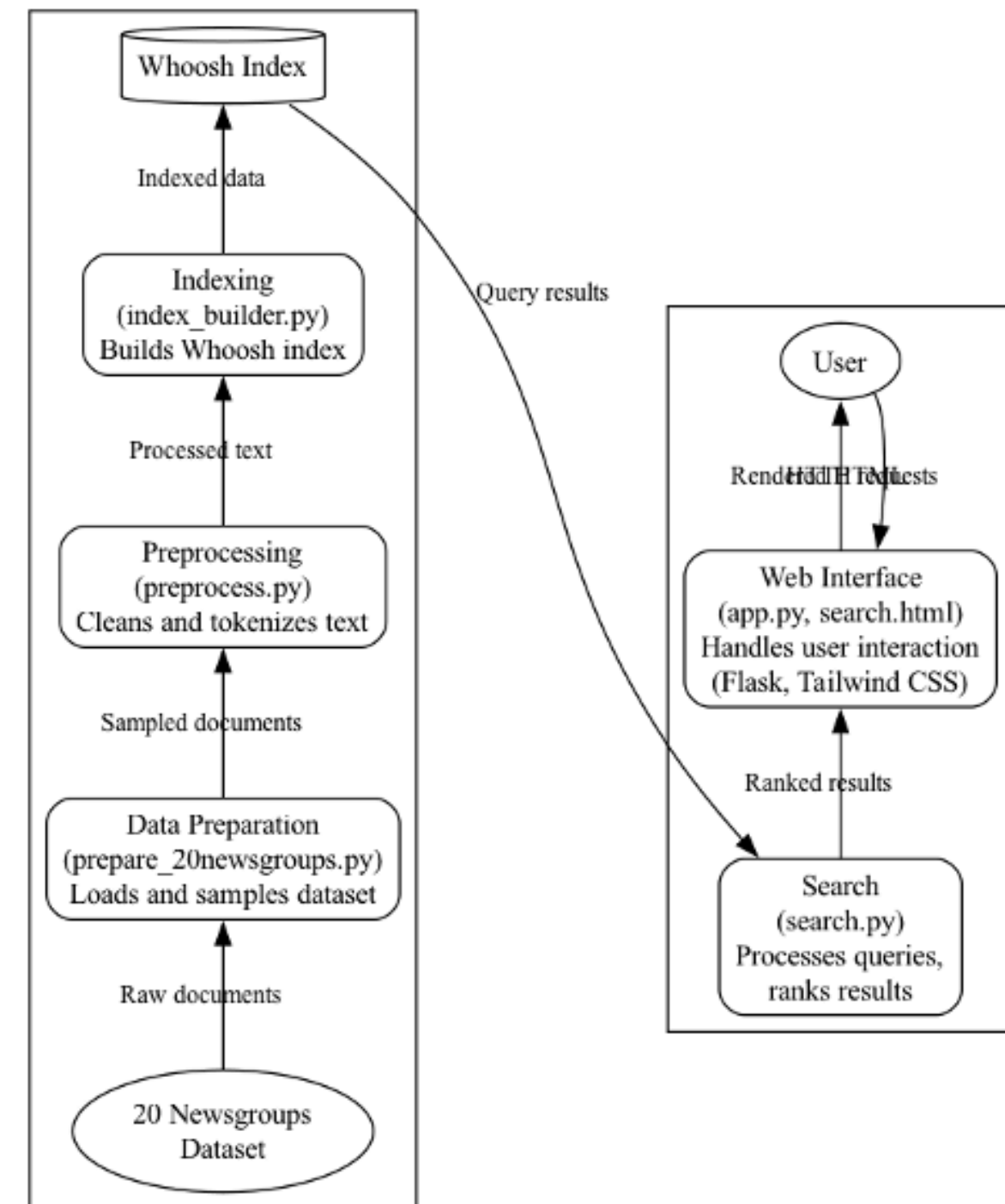
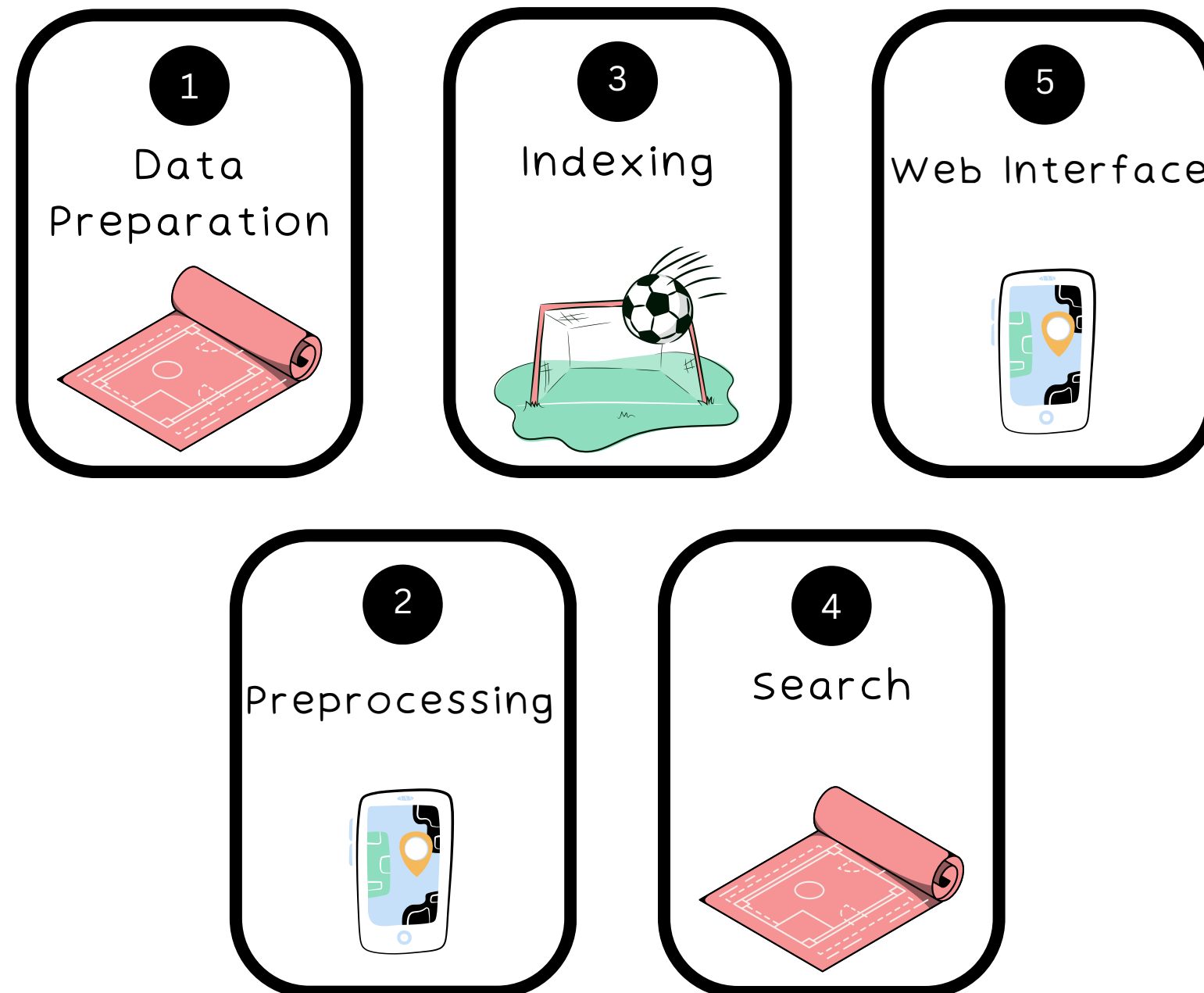
2. BM25

3. Custom Proximity-Based Approach

- Index the full dataset with all 20 categories.
- Achieve sub-second search times ($< 0.2s$).
- Provide a user-friendly web interface with features like pagination, category filtering, and autocomplete.
- Validate effectiveness through user feedback and performance metrics.

SYSTEM OVERVIEW

- High-Level Architecture:

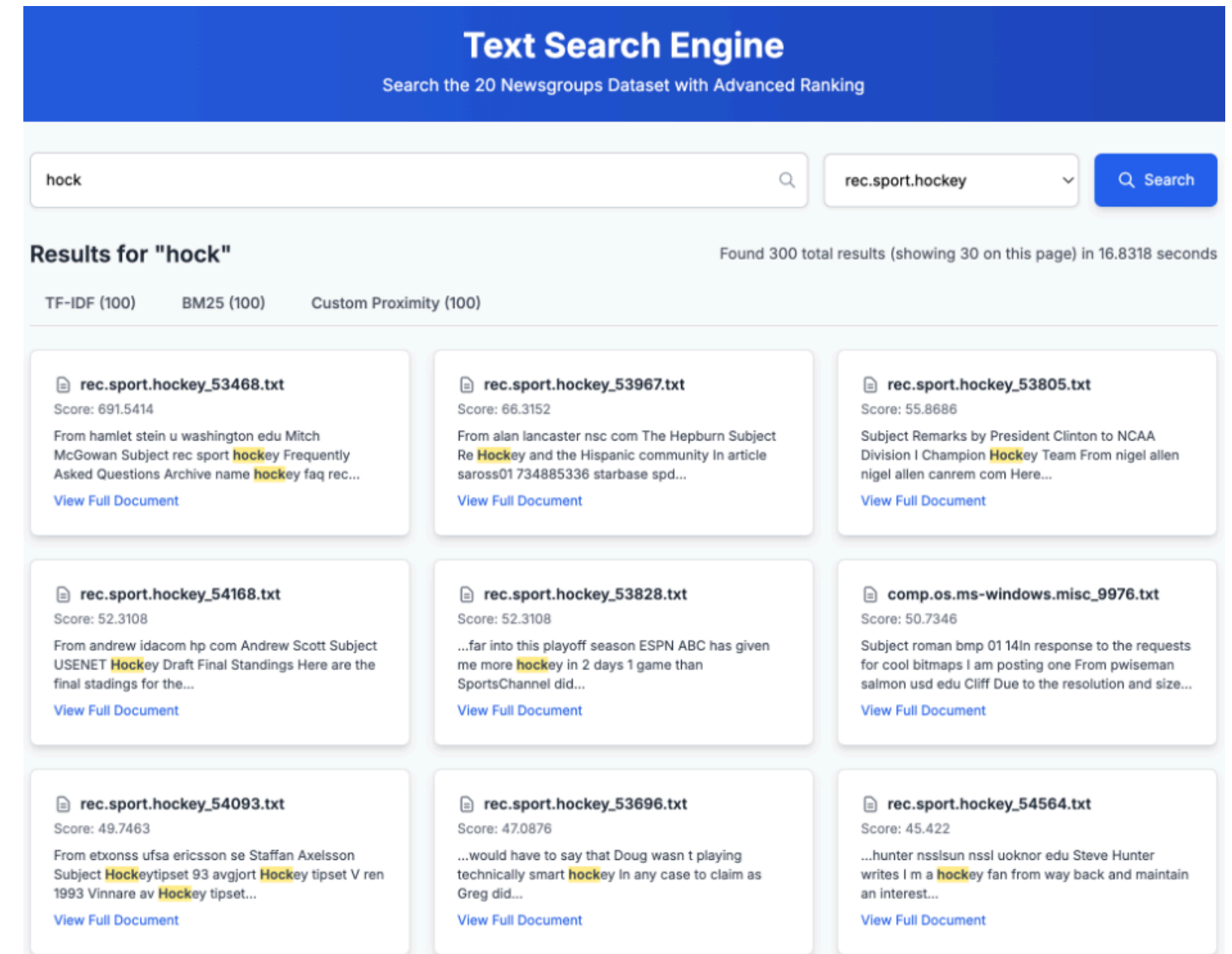


TECHNOLOGIES USED

- Backend:
- Python 3.9
- Whoosh (Indexing)
- NLTK (Text Preprocessing)



- Frontend:
- Flask (Web Framework)
- Tailwind CSS (Styling)
- JavaScript (Dynamic Features)



IMPLEMENTATION - RANKING METHODS

- Three Ranking Methods:
 - TF-IDF: Weights terms based on frequency and rarity.
 - BM25: Improves with term saturation and length normalization.
 - Custom Proximity: Scores higher for closer query terms (e.g., 'computer graphics').
-
- Example: Query 'computer graphics' in comp.graphics category:
 - TF-IDF: Top doc might prioritize frequency.
 - BM25: Balances frequency and doc length.
 - Custom: Prioritizes docs where "computer" and "graphics" are closer together.

IMPLEMENTATION – KEY CHALLENGE

Challenge: Incomplete Category Coverage:

- Initial Issue: Only 11/20 categories indexed due to document limit.
- Solution: Proportional sampling in `prepare_data`.

✓ All Categories
alt.atheism
comp.os.ms-windows.misc
comp.sys.mac.hardware
rec.autos
rec.motorcycles
rec.sport.baseball
rec.sport.hockey
sci.crypt
sci.med
talk.politics.mideast
talk.politics.misc

After: All 20 categories validated.

Before: 11 categories in dropdown.

✓ All Categories
alt.atheism
comp.graphics
comp.os.ms-windows.misc
comp.sys.ibm.pc.hardware
comp.sys.mac.hardware
comp.windows.x
misc.forsale
rec.autos
rec.motorcycles
rec.sport.baseball
rec.sport.hockey
sci.crypt
sci.electronics
sci.med
sci.space
soc.religion.christian
talk.politics.guns
talk.politics.mideast
talk.politics.misc
talk.religion.misc

CODE HIGHLIGHT: PROPORTIONAL SAMPLING

```
def prepare_data(max_docs=None, categories=None):
    dataset = fetch_20newsgroups(subset='all', categories=categories, remove=('headers', 'footers', 'quotes'))
    docs = []
    category_counts = {}
    for i, (content, category_idx) in enumerate(zip(dataset.data, dataset.target)):
        category = dataset.target_names[category_idx]
        category_counts[category] = category_counts.get(category, 0) + 1
        doc_id = f"doc_{i}"
        docs.append((doc_id, content, category))

    if max_docs and len(docs) > max_docs:
        total_docs = len(docs)
        docs_per_category = {cat: int((count / total_docs) * max_docs) for cat, count in category_counts.items()}
        sampled_docs = []
        for cat in docs_per_category:
            cat_docs = [(doc_id, content, c) for doc_id, content, c in docs if c == cat]
            sampled_docs.extend(cat_docs[:docs_per_category[cat]])
        docs = sampled_docs

    return docs
```

Code Snippet from prepare_data (prepare_20newsgroups.py)

IMPLEMENTATION: QUERY SUGGESTION

Feature: Query Suggestion for Misspellings

- Example: "mural network" → "neural network".
- Implementation: Used pyspellchecker and similarity matching.
- UI: Displays "These are results for neural network. Search instead for mural network."

The screenshot displays a web interface for a "Text Search Engine" that searches a "20 Newsgroups Dataset with Advanced Ranking". The search bar contains the query "mural network". To the right of the search bar is a dropdown menu labeled "All Categories" and a blue "Search" button. Below the search bar, a light blue banner displays a query suggestion: "These are results for [neural network](#). Search instead for [mural network](#)." Below this banner, the results for "mural network" are shown, indicating "Found 300 total results (showing 30 on this page) in 2.1498 seconds". The results are organized into three columns, each with a header: "TF-IDF (100)", "BM25 (100)", and "Custom Proximity (100)". Each column contains three search results, each with a document title, a score, a snippet of text, and a "View Full Document" link. The first result in the TF-IDF column is "comp.windows.x_68012.txt" with a score of 229.8923. The first result in the BM25 column is "sci.crypt_15177.txt" with a score of 170.1203. The first result in the Custom Proximity column is "sci.crypt_15176.txt" with a score of 151.7289.

Text Search Engine
Search the 20 Newsgroups Dataset with Advanced Ranking

mural network Q All Categories Q Search

These are results for [neural network](#). Search instead for [mural network](#). ×

Results for "mural network" Found 300 total results (showing 30 on this page) in 2.1498 seconds

TF-IDF (100) BM25 (100) Custom Proximity (100)

TF-IDF (100)	BM25 (100)	Custom Proximity (100)
comp.windows.x_68012.txt Score: 229.8923 ...list of X servers that can be used on nonUNIX sic networked machines to display X clients running on UNIX... View Full Document	sci.crypt_15177.txt Score: 170.1203 ...and other privacy and rights issues associated with use of the Internet and global networks in general Search for for... View Full Document	sci.crypt_15176.txt Score: 151.7289 ...rights issues associated with use of the Internet and global networks in general Search for for exact section Search for... View Full Document
comp.os.ms-windows.misc_10074.txt Score: 114.9461 ...each workstation hard drive assuming the resources are available From a network administrator s implementation he she would install all... View Full Document	sci.med_59154.txt Score: 70.7078 ...Mian K Sjolander D Haussler 3 00 3 30pm Break 3 30pm Protein Classification using Neural Networks E A Ferran... View Full Document	comp.graphics_38938.txt Score: 66.1099 From sull focus csl uiuc edu Sanghoon Sull Subject Final call for papers CALL FOR PAPERS Progress In Neural Networks... View Full Document
comp.sys.mac.hardware_51892.txt Score: 64.3698 ...this list possible Contents CPUs Hardware Monitor support Expansion Operating system OS Number Crunchina Networkina Printing The CPUs Note I...	comp.os.ms-windows.misc_10184.txt Score: 50.5763 ...Novell and Windows 3 11 m working at a workstation which is usually attached to a Novell network using...	comp.graphics_38976.txt Score: 49.0852 ...and Circuits COURSE Computer Network Architecture Pulse and Digital Circuits WORK Advanced Computer Communications Artificial

LIVE DEMO

Text Search Engine

Search the 20 Newsgroups Dataset with Advanced Ranking

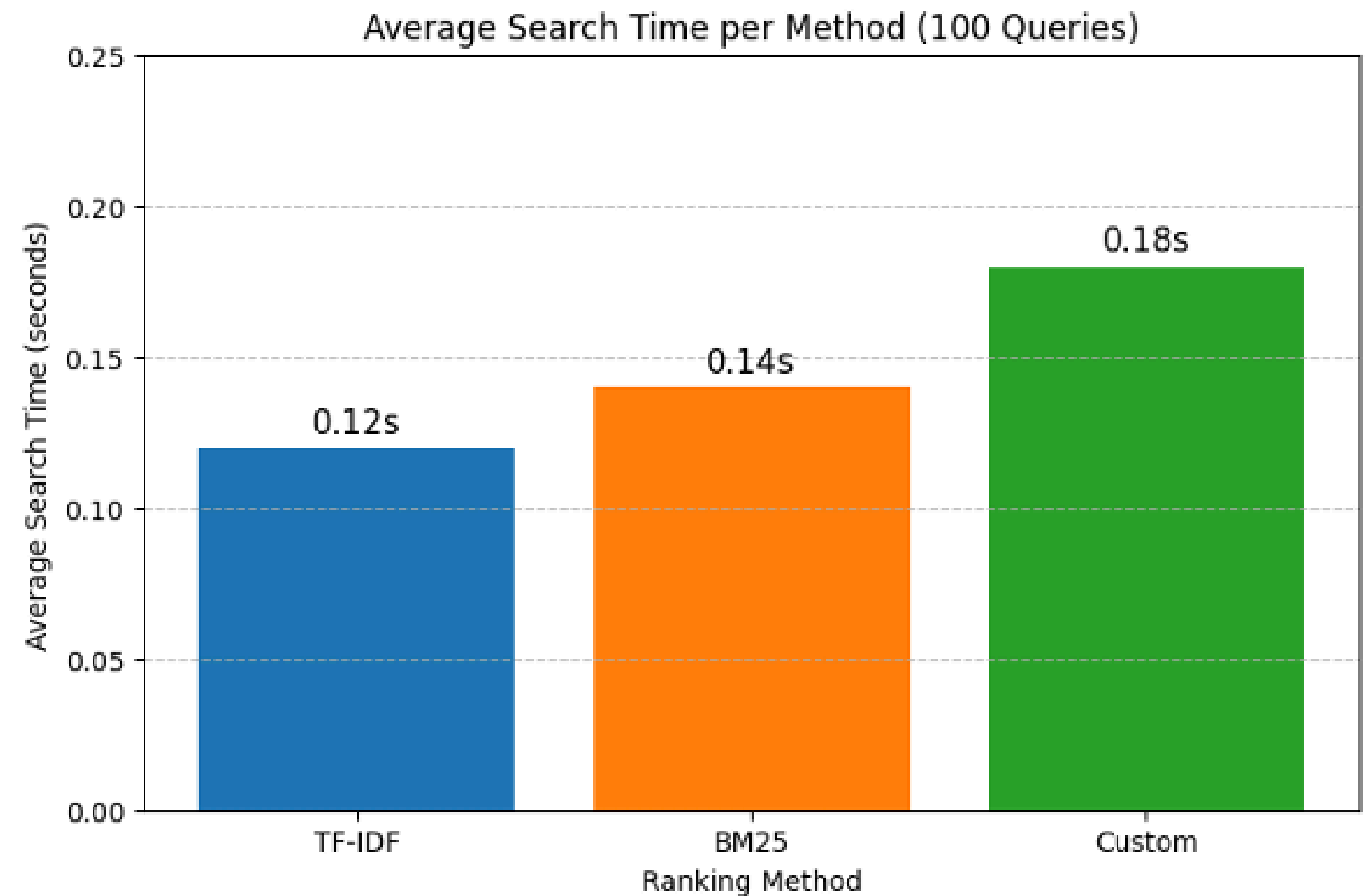
All Categories

▼

- Let's see the search engine in action!
- Features to showcase:"
- Search with "computer graphics" (compare TF-IDF, BM25, Custom).
- Category filtering (e.g., comp.graphics).
- Pagination (10 results per page).
- Autocomplete (e.g., typing 'comp').
- Query suggestion (e.g., 'mural network' → 'neural network').
- Document view (e.g., rec.sport.hockey_53967.txt).

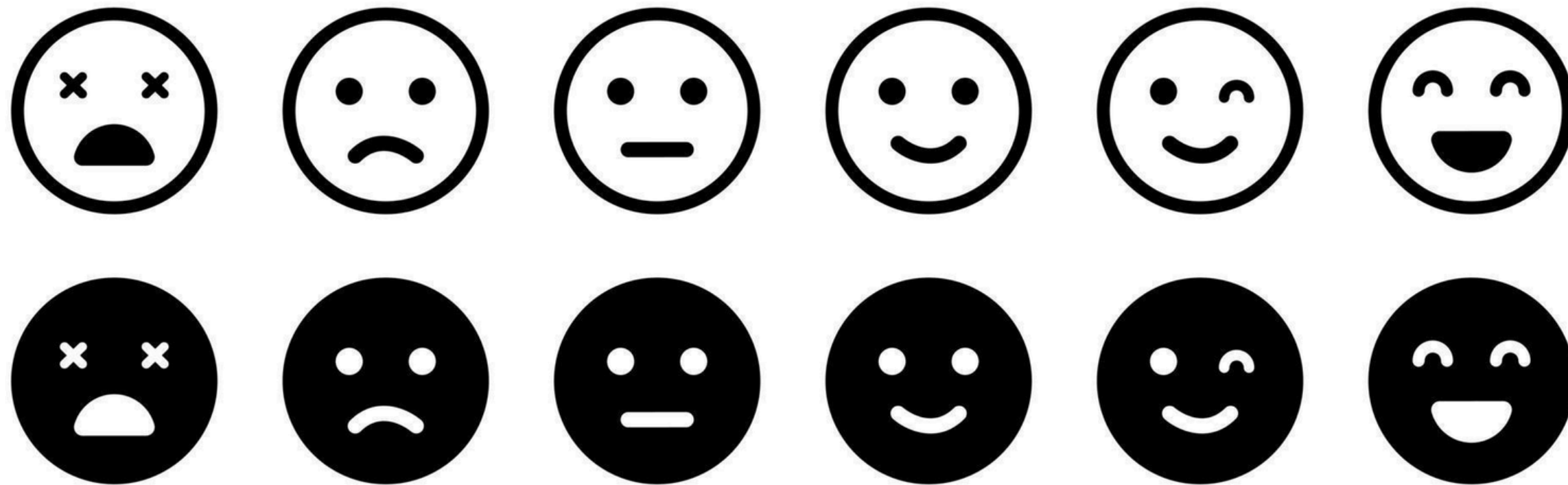
RESULTS: PERFORMANCE METRICS

- Search Times (avg):"
- TF-IDF: 0.12s
- BM25: 0.14s
- Custom: 0.18s
- Indexing Time: 25s for 18,846 documents
- Memory Usage: ~150 MB

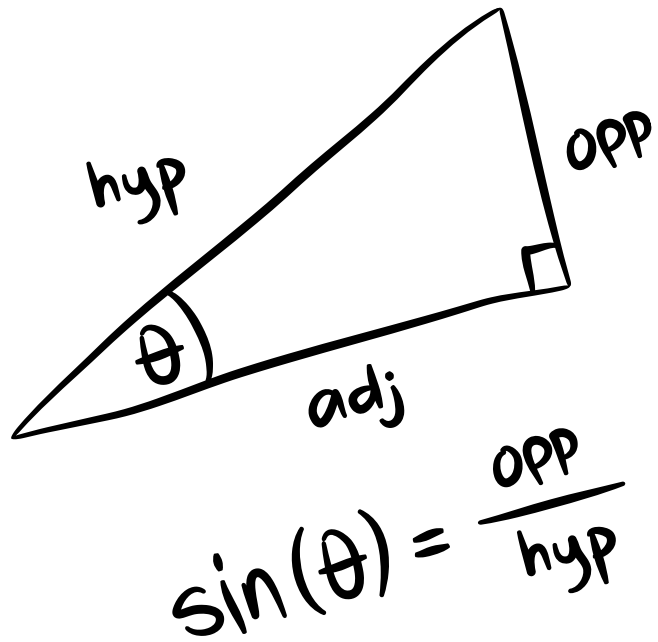


RESULTS: USER STUDY

- User Study (10 Participants):
- Avg. Time to Find Relevant Documents: 15 seconds.
- Feedback: Appreciated category filter, tabbed interface.
- Performance Target Achieved: All search times < 0.2s.



SUMMARY

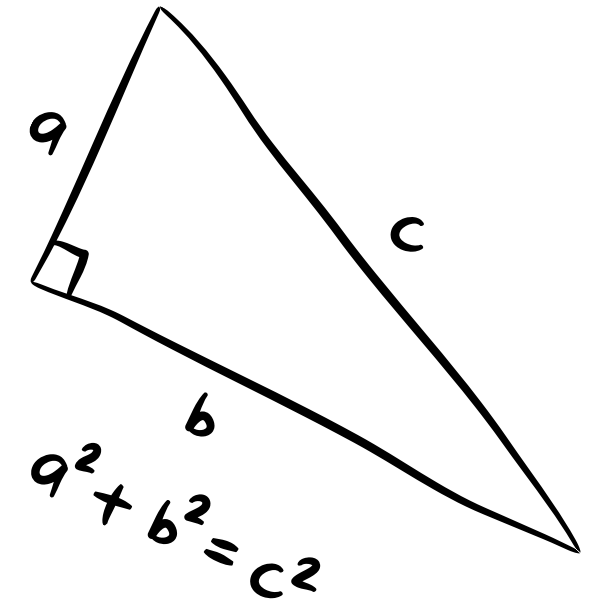


Contributions:

- Delivered a search engine for 20 Newsgroups with three ranking methods.
- Valuable tool for IR education in networked environments.

Limitations:

- Lacks advanced query syntax (e.g., wildcards, phrase queries).



$$y - y_1 = m(x - x_1)$$

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$



Thank You

